

CI/CD Benchmark & Optimisation Report

GitHub Actions to GitHub Pages | Pipeline Observatory | 30 September 2026

-13.3%

Mean duration

108.0s to 93.6s

-15.4%

p95 duration

117s to 99s

0%

Failure rate

10/10 passed

-84.3%

Rollback time

108s proxy to 17s

Objective and method

A real Vite website was built, tested, security-scanned and deployed to GitHub Pages. Two matched samples used five sequential workflow_dispatch runs on the same application, lockfile and ubuntu-latest runner label. Total duration is GitHub's workflow startedAt-to-updatedAt interval; p95 uses nearest rank. Step timestamps have one-second resolution.

Workflow duration (seconds)



Optimisations implemented

1	Dependency caching setup-node restores npm cache keyed by package-lock.json.
2	Parallel quality gates Lint/tests, CodeQL/dependency scans and build execute concurrently.
3	Build once and reuse The tested dist artifact is deployed directly and retained for 30-day rollback.

Measured results

Metric	Baseline	Optimised	Change
Average total	108.0 s	93.6 s	-13.3%
p95 total	117 s	99 s	-15.4%
Build	1.4 s / 2 builds	1.2 s / 1 build	1 build removed
Test	1.2 s	1.2 s	0%
Scan	68.6 s	65.4 s	-4.7%
Deployment job	19.0 s	9.4 s	-50.5%
Failure rate	0/5	0/5	No regression

Interpretation

Average delivery time fell by 14.4 seconds (13.3%) and p95 fell by 18 seconds (15.4%). The deployment job improved most because it no longer repeats checkout, dependency installation or compilation. CodeQL remained the critical path, so the total reduction is smaller than the 50.5% deployment-job improvement.

All ten measured runs passed, so failure rate remained 0%; an improvement could not be demonstrated without manufacturing failures. The automatic post-merge run (111 seconds) warmed the cache and is disclosed but excluded from the matched five-run sample. Aggregate npm ci time did not decrease because three parallel jobs each install dependencies; for this small project, runner variability dominates.

Rollback validation

Baseline recovery required rebuilding a historical commit through the full 108-second pipeline proxy. The new workflow redeployed the retained immutable artifact in **17 seconds** and the live-site smoke test passed. This is an **84.3% recovery-time reduction** and avoids dependency drift during recovery.

Evidence and traceability

Change	Baseline commit 95f5222	Optimisation PR #1
Baseline runs	1 / 2 / 3	4 / 5
Optimised runs	1 / 2 / 3	4 / 5
Rollback	Run 36704893909	Rollback workflow
Exports	Baseline CSV/JSON	Optimised CSV/JSON
Deployment	Live GitHub Pages site	Repository

Conclusion

The optimised workflow achieved repeatable mean and p95 reductions without removing any test or security gate. Artifact reuse delivered the clearest stage improvement and transformed rollback from a full rebuild into a fast, verified redeployment.

Limitations: n=5 per phase; GitHub-hosted runner demand varies; one-second timestamps obscure subsecond changes; the 108-second baseline rollback value is a full-pipeline proxy rather than a separately repeated rollback sample.